

ORGANIZATION OF DIGITAL COMPUTERS LAB EECS 112L

Lab #01

Student Name: STEVEN KUZHIPALA

Student ID #: 51378962

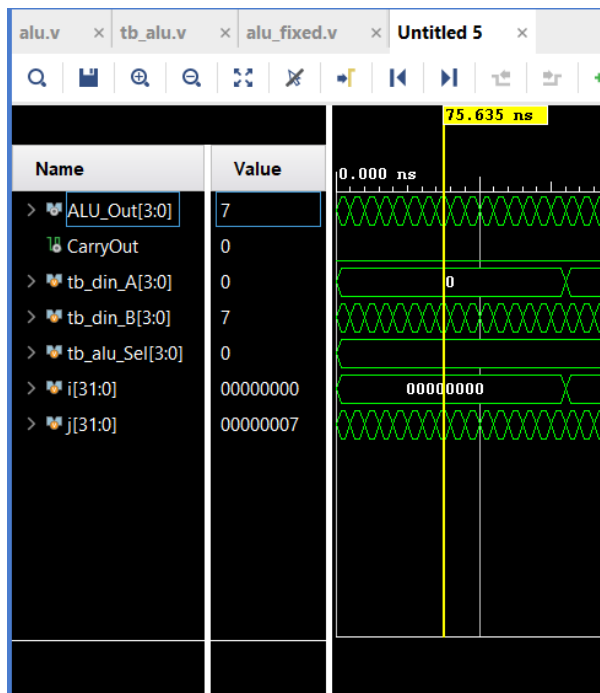
Verification of Combinational Logic – ALU

1. ALU Addition:

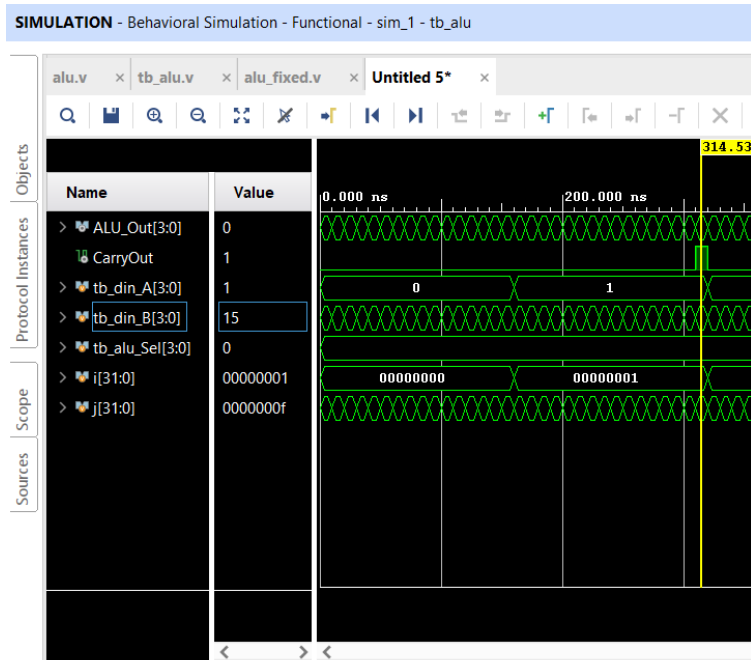
The provided Addition operation in alu.v was correct in terms of its output, and the carryout as well. Whenever the ALU_Out was 16, the carryout bit was high. However, carryout did end up going high for all operations, which is a problem. Carry out was separately fixed for its output in other operations and is explained later in the report.

Some of the successful addition operation waveforms and their explanation are given below:

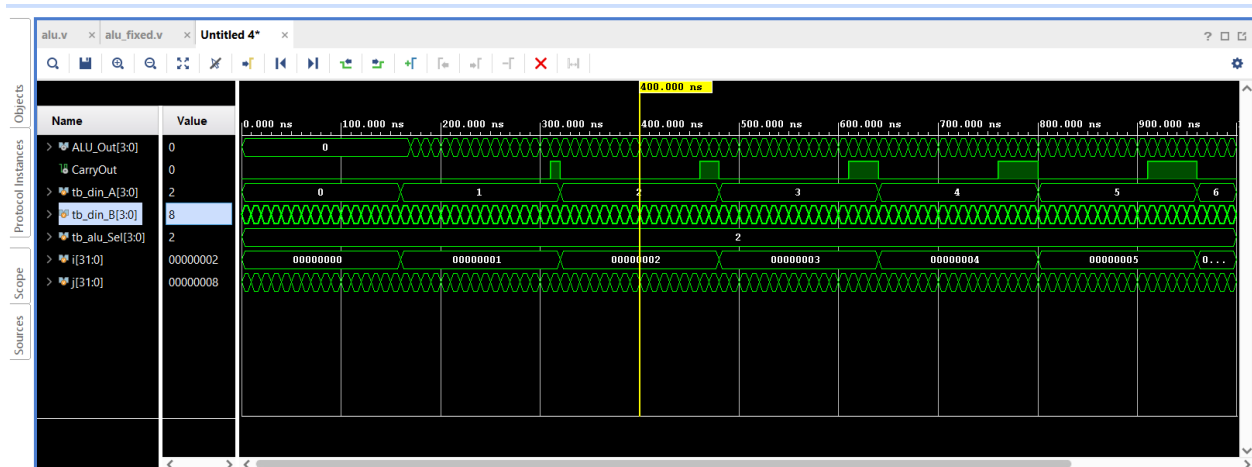
1) In this first example, the addition of input A=0 and the input B=7, has been correctly computed to sum 7, with no carryout, which is what we expected.



2) In this second example, the sum of $A=1$. And $B=15$, has been computed correctly to be 16, along with one carryout bit, also shown as high appropriately on the waveform.

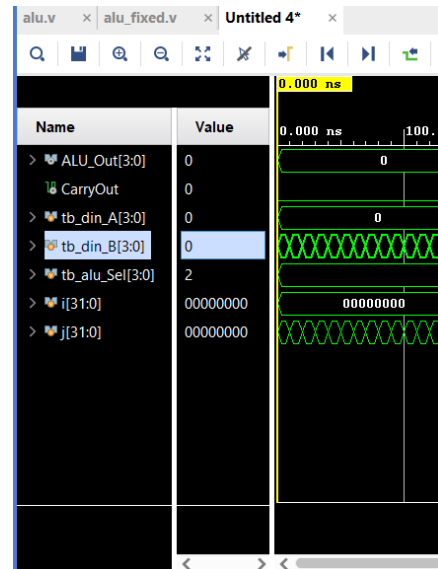


2. ALU Multiplication:



The multiplication process appears to be fine itself in terms of the result being shown on ALU_Out which only shows 4 bits (the full result is placed in ALU_Result). Here the 2×8 does result in 16 which requires 5 bits, but because ALU_Out is only 4 bits, the last 1 bit is to be set to be a carryout (which it ISN'T).

Another example for a correct MULTIPLICATION output for the ALU multiplication is given to the right, the classic 0×0 , which outputs a 0 as expected.



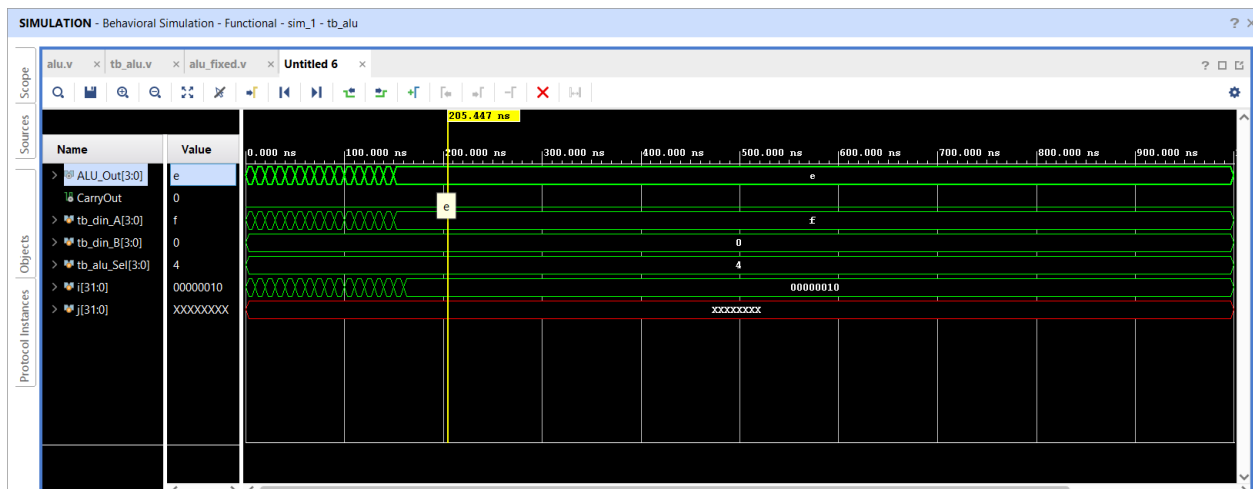
The carryout is not correct for this operation, as it is still being shown as the carry of $A+B$, and note the carry from $A*B$.

The exact line from the UNFIXED alu.v file that shows the incorrect carryout is shown here:

```
assign CarryOut = tmp[4]; // Carryout flag
```

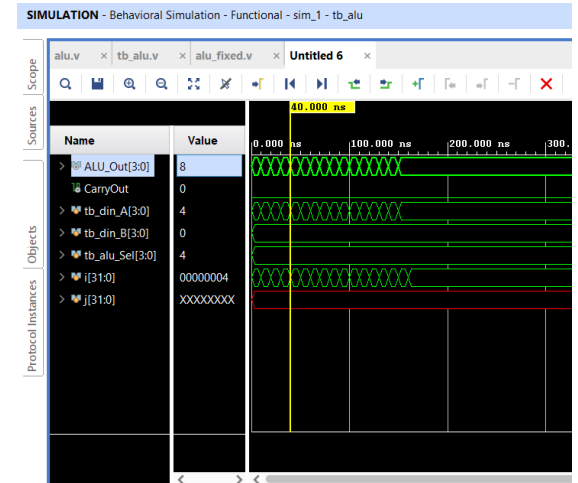
The fix for this issue is to follow the lab manual's suggestion and switch the carry out to be set to be 5th bit of the internal result.

3. ALU Logical Shift Left:



Similar to the multiplication problem, the output for the Logical Shift Left operation seems to be correct, but the output of the carryout is incorrect. In this example, when shifting A's value of 16 by one bit to the left (B's is left unused in the shifting and rotating operations), we get the right output of e (15) in ALU_Out, but the carryout should have been 1 and not 0.

Another example to the right shows us shifting A again, starting with a value of 4 which, when shifted to the left, correctly outputs the number 8, with no carryout issue.



The carryout is not correct for this operation, as it is still being shown as the carry of A+B, and note the carry from shifting A to the left.

The exact line from the UNFIXED alu.v file that shows the incorrect carryout is shown here:

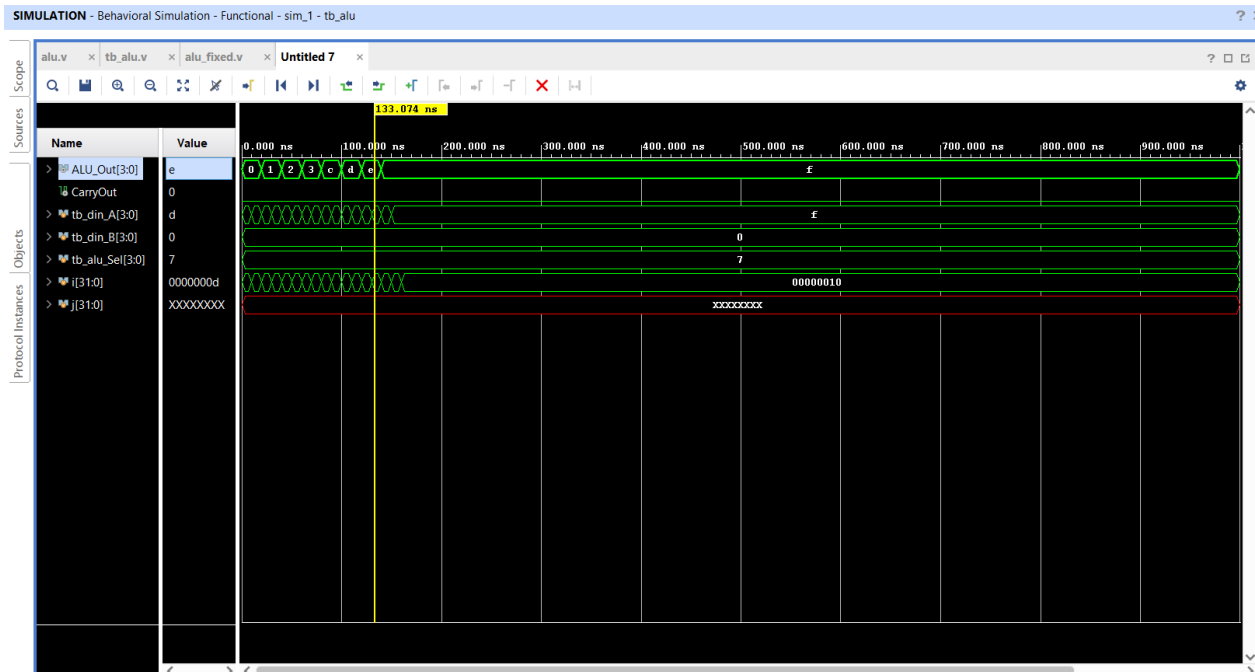
```
assign tmp = (A << B) | (A >> B);  
assign CarryOut = tmp[4]; // Carryout flag  
// ...
```

The fix for this issue is to follow the lab manual's suggestion and switch the carry out to be set to be the bit that jumps out of the 4 bit width of ALU_Result.

4. ALU Rotate Right:

The ALU Rotate right operation appears to be INCORRECT in the alu.v file based of the output below.

The value of A is set to be d (which is 13) and after the rotate right operation, it must change into 11 (b), however it is shown be to be e (14), which is incorrect.



The exact line causing this problem is given below:

```
4'b0111: // Rotate right
    ALU_Result = {A[3],A[3:1]};
```

The way to fix this issue is by changing the most significant bit to be set to ALU[0] instead of ALU [3] , as this would allow the bottom of the original number to wrap up to the top, as shown below, and in the fixed alu file:

```
4'b0111: // Rotate right
    ALU_Result = {A[0], A[3:1]};
```

The carryout is not correct for this operation, as it is still being shown as the carry of A+B, and note the carry from rotating A to the right.

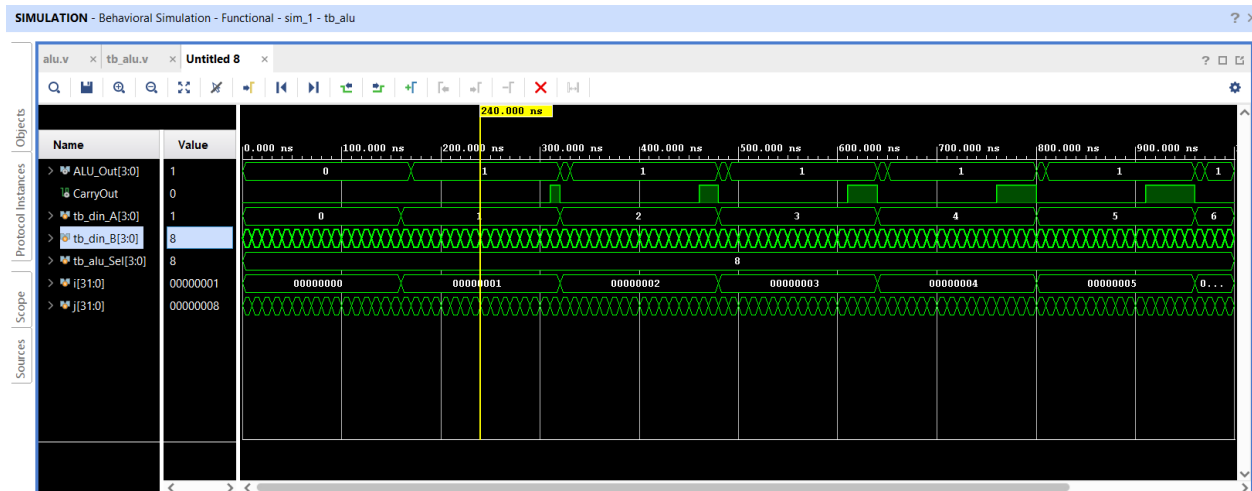
The exact line from the UNFIXED alu.v file that shows the incorrect carryout is shown here:

```
assign CarryOut = tmp[4]; // Carryout flag
```

The fix for this issue is to follow the lab manual's suggestion and switch the carry out to be set to be the bit that jumps out of the 4-bit width of ALU_Result.

5. ALU Bitwise AND:

The ALU Bitwise AND operation appears to be INCORRECT in the alu.v file based of the output below.



Here As input is 1 and B is 8, and after a bitwise AND operation, the result must be 0, however the ALU_Out shows a 1 which is incorrect. Also, in this operation the carryout shouldn't ever be high, which is also another incorrect output for this design in the original alu.v file.

The exact line from the UNFIXED alu.v file that shows the incorrect logical operation is shown here:

```
4'b1000: // Bitwise and
    ALU_Result = A && B;
```

This is a simple fix, and it's to only use one & operator instead of two, to make the operation into a bitwise AND, instead of a Logical AND.

```
4'b1000: // Bitwise and
    ALU_Result = A & B;
```

The carryout is not correct for this operation, as it is still being shown as the carry of A+B, instead of just avoiding a carry entirely for this logical operation.

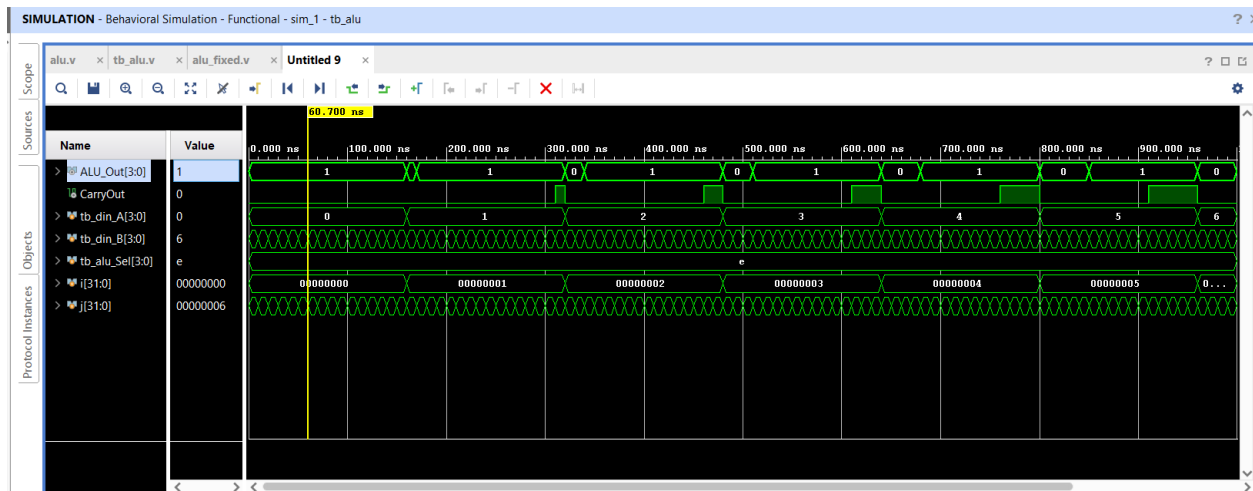
The exact line from the UNFIXED alu.v file that shows the incorrect carryout is shown here:

```
assign CarryOut = tmp[4]; // Carryout flag
```

The fix for this issue is to follow the lab manual's suggestion and switch the carry out to be set to be the bit that jumps out of the 4-bit width of ALU_Result.

6. ALU Greater Than Comparison:

The ALU Greater Than Comparison operation appears to be INCORRECT in the alu.v file based on the output below:



Here, even though A has a value of 0, and B has a value of 6, the $(A > B)$ output is labelled as HIGH, which is not true.

The exact line from the UNFIXED alu.v file that shows the incorrect logical operation is shown here:

```
4'b1110: // Greater comparison
ALU_Result = (A>B) ? 4'b0 : 4'b1;
```

This is a simple fix, and it's to only use one & operator instead of two, to make the operation into a bitwise AND, instead of a Logical AND.

```
4'b1110: // Greater comparison
ALU_Result = (A>B) ? 4'b1 : 4'b0;
```

The carryout is not correct for this operation, as it is still being shown as the carry of A+B, instead of just avoiding a carry entirely for this logical operation.

The exact line from the UNFIXED alu.v file that shows the incorrect carryout is shown here:

```
assign tmp[4] = (A < B) ? 1 : 0;
assign CarryOut = tmp[4]; // Carryout flag
// ...
```

All the carryout's have been fixed individually in the alu_fixed file, and some of them have been now avoided as requested in the lab manual:

```
12
13 assign CarryOut = (ALU_Sel == 4'b0000) ? tmp[4] :           // Addition
14                   (ALU_Sel == 4'b0001) ? (A < B) :         // Subtraction
15                   (ALU_Sel == 4'b0010) ? ALU_Result[4] :   // Multiplication
16                   (ALU_Sel == 4'b0011) ? ALU_Result[4] :   // Division
17                   (ALU_Sel == 4'b0100) ? A[3] :            // Logical shift left
18                   (ALU_Sel == 4'b0101) ? A[0] :            // Logical shift right
19                   (ALU_Sel == 4'b0110) ? A[3] :            // Rotate left
20                   (ALU_Sel == 4'b0111) ? A[0] :            // Rotate right
21                   1'b0;                                     // everything else is 0
22
23 always @(*)
```

Verification of Sequential Logic - D flip-flop

1. D Flip-Flop with Synchronous Set/Reset:

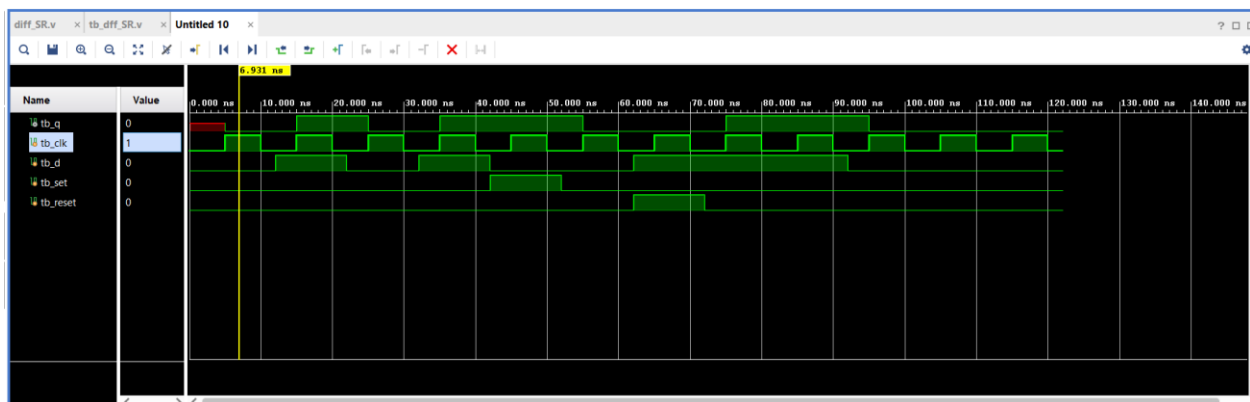
For a synchronous D Flip-Flop, q must copy d's value, only at the rising edge of the clock, and if the set or reset inputs go high, q MUST NOT change its value till the next rising edge.

If the set bit is also set HIGH, the output of Q is set to 1.

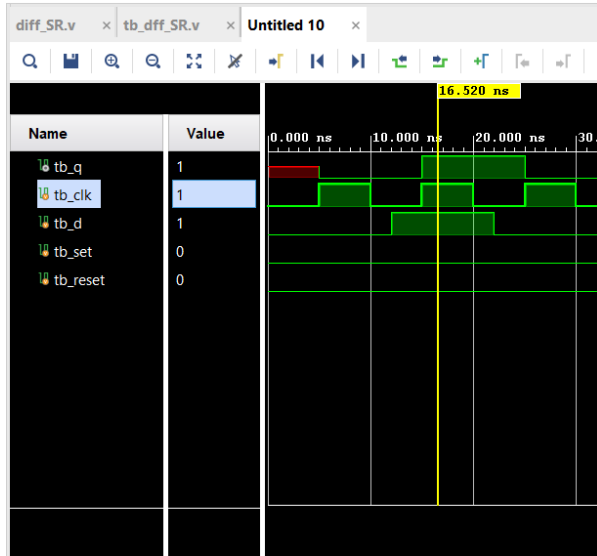
The following table was created from the **appendix of the lab manual** to verify the simulation results:

Set	Reset	Clock (clk)	Data (D)	Next State (Q)	Description / Operation
0	0	(Active edge)	0	0	Samples D (0) on the active clock edge

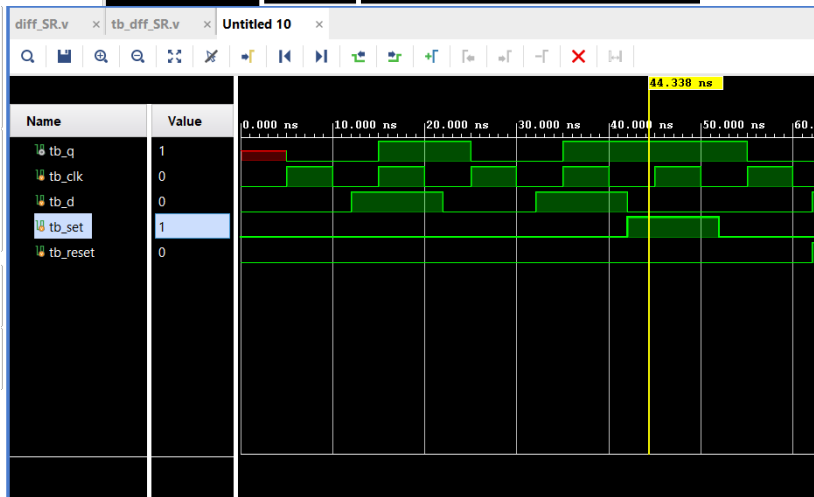
Set	Reset	Clock (clk)	Data (D)	Next State (Q)	Description / Operation
0	0	(Active edge)	1	1	Samples D (1) on the active clock edge
1	0	X (Ignored)	X (Ignored)	1	Output Q is forced to 1, regardless of clock or D
0	1	X (Ignored)	X (Ignored)	0	Output Q is forced to 0, regardless of clock or D
1	1	X	X	Invalid / Illegal	Illegal combination of inputs (avoided in tests as requested in lab manual)



We can see here right to the left of the yellow cursor, that the right at the clk's rising edge, q 's value changes from being unknown to be 0, which matches the input d .

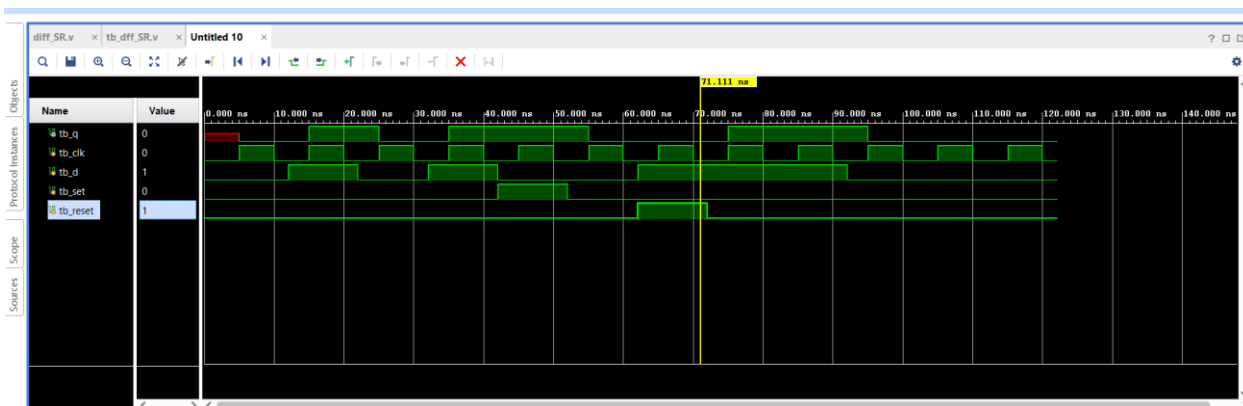


In this image to the left, we can see how when d's value changes, and the clk's rising edge goes high again, with set and reset both set to 0, q changes its values again at the rising edge of clock to be 1, matching d's value. This matches the synchronous set up for this D-flip-flop.



In this image, you can see how when the set bit is set to HIGH, regardless of the rising edge of the clock, Q is set to the value of 1 and holds the value till set goes down and till the next rising clock edge, when it hits zero (as seen to the right of the yellow line).

Lastly, as shown in the images below, you can see that when the reset bit is set to be HIGH, q's value is set to 0, regardless of the clock's rising edge, and it only change back to 1 (from d's value) in the next rising edge of the clock, and when reset is set back to 0.

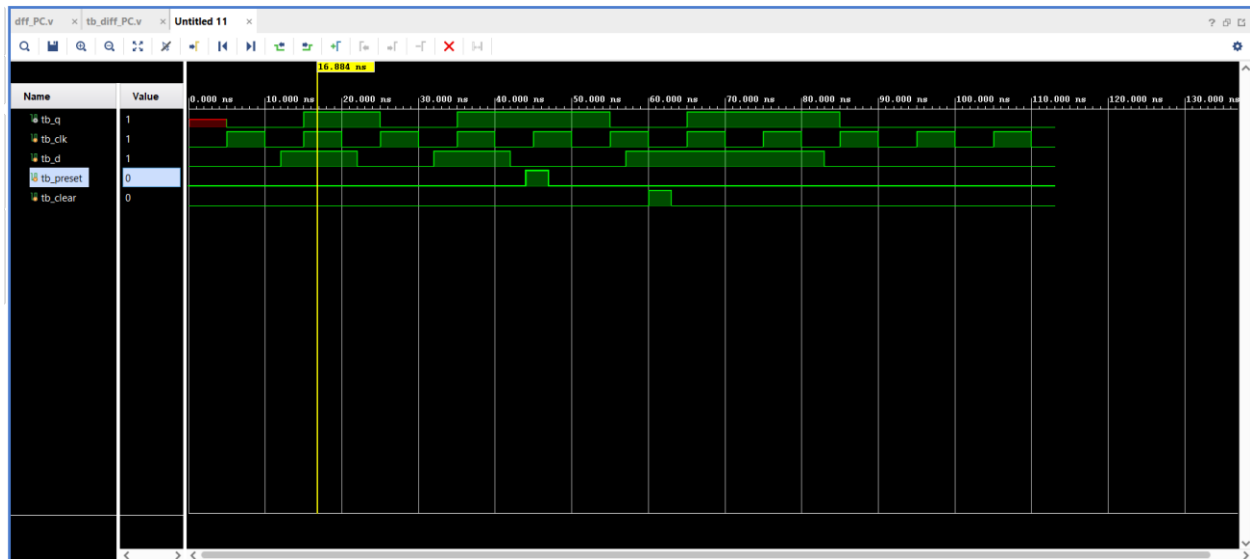


2. D Flip-Flop with Asynchronous Preset/Clear:

This kind of flip-flop has a similar operation to a Synchronous D flip-flop, but whenever the preset or clear bit goes HIGH, q changes its value immediately without the need for rising edge of the clock.

The following table was created from the **appendix of the lab manual** to verify the simulation results:

Preset	Clear	Clock (clk)	Data (D)	Next State (Q)	Description / Operation
0	0	(Active edge)	0	0	Samples D (0) on the active clock edge
0	0	(Active edge)	1	1	Samples D (1) on the active clock edge
1	0	X (Ignored)	X (Ignored)	1	Output Q is forced to 1, regardless of clock or D
0	1	X (Ignored)	X (Ignored)	0	Output Q is forced to 0, regardless of clock or D
1	1	X	X	Invalid / Illegal	Illegal combination of inputs (avoided in tests as requested in lab manual)



As shown in the image above, with preset and clear set to low, q changes its value according to the rising clock edge, and follows the same value as input d. However, at about the 43.0 ns mark, preset turns high, and despite the change of tb_d's value from HIGH to LOW, q remains to stay positive throughout, as preset is high.

Similarly, at the 60.0 ns mark, when clear is set to HIGH, q is set to be low despite the HIGH value for tb_d, and the rising clock edge.

Midterm Question Winter 2022:

```

1 module ctr8 (
2     output reg [7:0] counter,
3     output reg carryout,
4     input [7:0] data, input load, clk, reset );
5
6     always @(posedge clk or negedge reset)
7     begin
8         if (!reset) {carryout, counter} <= 9'b0;
9         else if (load) {carryout, counter} <= data;
10        else {carryout, counter} <= counter + 1'b1;
11    end
12 endmodule
13

```

For this solution, we only need to add a negedge reset as another signal in our sensitivity list to make convert the module to a synchronous reset module, as the `if(!reset)` conditional already handles the reset on active low, and the load is also only set with a high clock, keeping it still synchronous as well.